


# La experiencia como producto

## Lecciones aprendidas

SPONSORS:

NTT DATA · FOUR9S · V-Valley · Coyote Monitor by Aleson ITC · encamina PIENSA EN COLORES · Next Step · prodware · TD SYNNEX · ALTIA · waibbo

## PLATINO

NTT DATA

FOUR9S

V-Valley

## gold

Coyote Monitor
by Aleson ITC

encamina
PIENSA EN COLORES

Next Step

prodware

TD SYNNEX

## silver

ALTIA

waibbo

# Premios + Data Beers incluidas




















# Tres trayectorias, una misma conclusión

**75+**

años de proyectos de datos entre los tres

**25+**

años cada uno, de los primeros data warehouses al lakehouse

**1**

framework que resume lo aprendido

**Casi nunca falla la tecnología.** Fallan las convenciones, la estructura, la calidad del dato y quién responde de cada pieza.

LA IDEA DEL WORKSHOP

> **La IA amplifica lo que encuentra en el repositorio. Si encuentra convenciones, las sigue. Si encuentra caos, lo multiplica.**

Por eso la IA Generativa no sustituye al marco de trabajo: lo necesita. Todo el workshop está construido sobre un repositorio que lleva las reglas: lo veréis en cada demo.

# Lo que te llevas hoy



**Repositorio plantilla**

El framework completo y las ramas alumno-99/demo-NN para repetir cada demo en casa



**Librería datafwk**

Ingesta, estandarización, calidad y auditoría para Fabric



**AGENTS.md y prompts**

Para que cualquier asistente de IA respete el marco



**Linters y CI**

Nada entra en main sin tests, contratos ni revisión



**Lecciones aprendidas**

Cada una traducida a una regla del framework

# Agenda · 16:00 – 20:00

| | | |
|---|---|---|
| **16:00** | **Apertura** | Bienvenida y comprobación de entorno |
| **16:10** | **Teoría** | T1 · Anatomía de un framework de datos |
| **16:30** | **Demo** | Demo 1 · Greenfield: un producto nuevo |
| **17:05** | **Demo** | Demo 2 · Carga completa y dimensión tipo1 (con T2) |
| **17:30** | **Demo** | Demo 3 · Errores de origen y cuarentena Q1 → Q4 (con T3) |
| **18:10** | **Descanso** | Descanso |
| **18:25** | **Demo** | Demo 4 · De carga completa a incremental |
| **18:55** | **Demo** | Demo 5 · SCD2: ciudad con historia |
| **19:40** | **Teoría** | T4 · La IA dentro del marco |
| **19:55** | **Cierre** | Las lecciones y el repositorio |

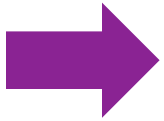
# El escenario: FraSoHome

### FraSoHome, retail de hogar
Volcado real: 49 tablas SQL y 10 contenedores Cosmos

### Devoluciones
397 líneas; cuatro defectos sembrados a propósito

### CRM
100 clientes; C0001 en Sevilla y la marca fecha_cambio

### Cosmos
Documentos JSON íntegros; sin pipeline hasta tener contrato



### `lh_bronze`
Lo que llegó, como llegó, con trazabilidad

### `lh_silver`
Limpio y conforme a contrato

### `lh_gold`
Modelo de consumo para negocio

CHECKPOINT 10 minutos Los tres

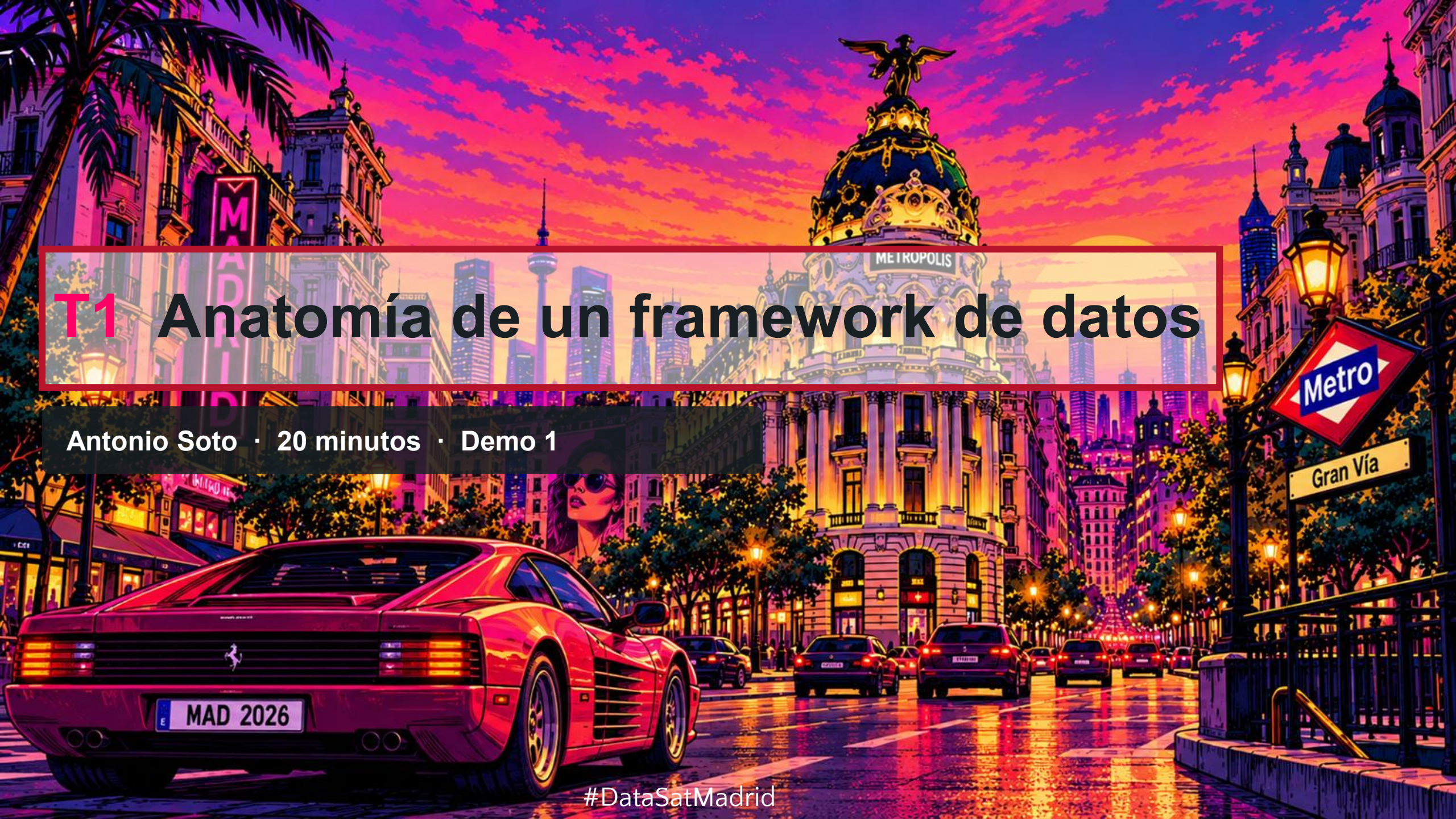
# Comprobación de entorno

1. .env.alumno cumplimentado y az account show en tu tenant, con la misma cuenta que el portal
2. Tres workspaces propios en tu capacidad y env_fwk en Runtime 1.3 (Spark 3.5)
3. python -m scripts.modificar_csv verificar termina bien
4. ¿Algo falla? Levanta la mano ahora, no en la demo 1

**Por qué ahora**

**Lo que no funcione ahora nos robará tiempo en cada demo.**

Capacidad pequeña (Trial, F2–F8): una sola sesión de Spark. Stop session antes de cada notebook.




# T1 Anatomía de un framework de datos

Antonio Soto · 20 minutos · Demo 1

#DataSatMadrid

LECCIÓN 1

# El proyecto se decide en la primera semana.

Las decisiones de nombres, capas y estructura que se toman con prisa el primer día son las que más cuesta deshacer el segundo año.

La cuenta Antonio: una batalla real

# Por qué fallan los proyectos de datos

Casi nunca por la tecnología. Casi siempre por esto:

### Convenciones ausentes

Cada uno nombra tablas, columnas y notebooks a su manera

### Lógica copiada

El mismo cálculo en diez sitios, cada uno con su variante

### Calidad sin dueño

El dato malo atraviesa todas las capas hasta el informe

### Despliegues que dependen de alguien

Si esa persona está de vacaciones, no se despliega

# Un framework son decisiones tomadas una vez

## Sin framework

- Cada notebook decide nombres y rutas
- Cada persona carga los datos a su manera
- El dato malo se descubre en el informe
- Desplegar depende de quien lo hizo

## Con framework

- Convenciones escritas y comprobadas
- Una librería común y configuración
- Contratos que deciden dónde aterriza el dato malo
- Despliegue automático desde main

# Las siete piezas del framework

### 1 · Convenciones

`docs/convenciones.md`

### 2 · Estructura

`Raíz del repositorio`

### 3 · Librería común

`src/datafwk/`

### 4 · Metadatos

`config/`

### 5 · Contratos

`contracts/`

### 6 · Testing y CI/CD

`tests/` · `.github/`

### 7 · Capa de IA

`AGENTS.md` · `prompts`

# Convenciones: se deciden una vez y se cumplen siempre

| Elemento | Convención | Ejemplo |
|---|---|---|
| Workspaces | `<proyecto>-<función>-<entorno>` | `fsh-alumno01-ing-dev` |
| Lakehouses | `lh_<capa>` | `lh_bronze · lh_silver · lh_gold` |
| Tablas silver | `<entidad> de negocio` | `clientes_crm · lineas_devolucion` |
| Tablas gold | `fact_<hecho>_<grano> · dim_<entidad>` | `dim_cliente · dim_cliente_actual` |
| Cuarentena | `fwk_cuarentena por versión` | `lote dev-q1 · contrato v2` |
| Columnas | `snake_case; técnicas con _` | `fecha_cambio · _batch_id` |
| Notebooks | `nb_<nn>_<capa>_<propósito>` | `nb_20_silver_estandarizar` |

# La estructura del repositorio

```
proyecto-datos/
├── AGENTS.md          # reglas para la IA
├── fabric/            # sincroniza Fabric
│   ├── datos/         # lakehouses
│   ├── ingenieria/    # notebooks
│   └── analitica/     # modelos, informes
├── src/datafwk/       # librería común
├── config/            # fuentes, entornos
├── contracts/         # contrato por tabla
├── templates/         # punto de partida
├── tests/
├── docs/              # glosario, ADR
└── .github/           # CI/CD y prompts
```

### fabric/ por workspace

Una carpeta por workspace: datos, ingeniería y analítica

### La lógica, en la librería

Los notebooks solo orquestan; la lógica tiene tests

### Lo que lee la IA

AGENTS.md y docs/glosario.md, antes que el código

# Tres workspaces y un solo camino a producción

**datos**

Lakehouses · sin código

**ingenieria**

Notebooks · pipelines · Environment

**analitica**

Modelos semánticos · informes

## Del cambio a producción

**Rama feat/…**

Workspace de feature conectado a Git

**Pull request**

CI: linters, tests, revisión

**main**

Despliegue automático

**Dev → Test**

fabric-cicd + tests de integración

**Prod**

Manual, con aprobación

▶ DEMO 1 min

# Fabric en Git: por qué la IA puede escribir notebooks

```
nb_20_silver_estandarizar.Notebook/
├── notebook-content.py
└── .platform

# notebook-content.py
# PARAMETERS CELL ****
entorno = "dev"
workspace_datos = "proyecto-datos-dev"
fuente = "pedidos"

# CELL ****
ctx = cargar_contexto(entorno, ...)
a_silver(spark, ctx.fuente(fuente), ...)
```



## Un .py legible

Celdas separadas por comentarios: se lee, se revisa y se compara



## logicalId único

Copiar un item sin cambiarlo confunde a Fabric (regla FW006)



## Código como cualquier otro

Un asistente en VS Code lo edita con las reglas del repo

DEMO 1 | 35 minutos | Lidera: Eladio

# Greenfield: un producto nuevo

1. crear_proyecto.py: destino nuevo y logicalId nuevos; sin .git ni dotenv
2. configurar · checkpoint base · preparar: contratos Q1 y tipo1, CSV de trabajo
3. Publica: lakehouses, wheel en env_fwk (6 min), notebooks, config y datos
4. Vincula los cinco notebooks a env_fwk y para las sesiones anteriores
5. Comprueba lh_bronze/Files/_framework y Files/landing
6. Mientras se publica el wheel: «cómo llega cada pieza a Fabric»

**La idea que se llevan**

**Contratos antes que código. El repositorio es la fuente de verdad; Fabric es donde se ejecuta.**

**Guía:** `docs/workshop/02_seguir-las-demos.md` · rama `alumno-99/demo-01`

# Cómo llega cada pieza a Fabric

| En el repositorio | Cómo viaja | Dónde aterriza en Fabric |
| --- | --- | --- |
| `src/datafwk/` | wheel | `Environment env_fwk` |
| `config/` · `contracts/` | publicar_config.py | `lh_bronze/Files/_framework/` |
| `fabric/<workspace>/` | fabric-cicd + parameter.yml | `Workspaces de Dev, Test y Prod` |



# T2 Ingesta guiada por metadatos

Miguel Egea · 15 minutos · Demo 2

LECCIÓN 2

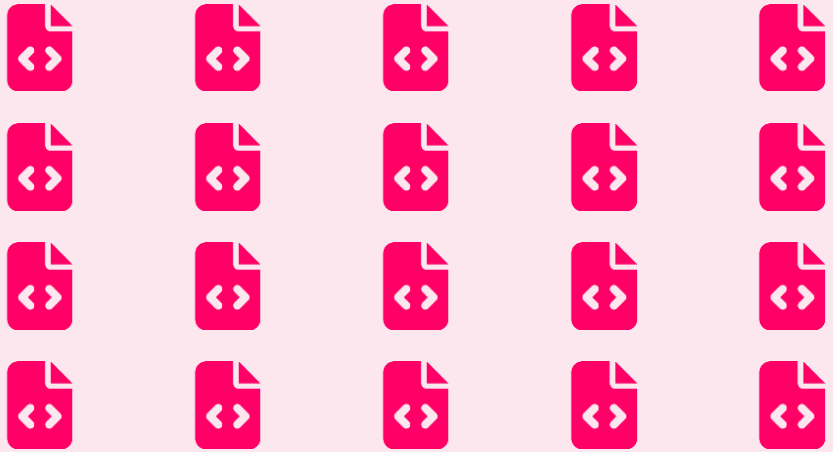
> **Lo que no está parametrizado se copia, y lo que se copia diverge.**

Un bug corregido en un notebook copiado sigue vivo en todos los demás.

La cuenta Miguel: una batalla real

# Un notebook genérico y configuración, no un notebook por fuente

## Un notebook por fuente



20 notebooks casi iguales: el bug corregido en uno sigue vivo en 19

## Dos notebooks + configuración

`nb_10_bronze_ingesta`

`nb_20_silver_estandarizar`

`config/fuentes/crm.yml`

Añadir una fuente es añadir unas líneas de YAML, no código

# Una fuente es configuración

```yaml
# config/fuentes/crm.yml
dominio: crm
fuentes:
  clientes_crm:
    formato: csv
    clave_negocio: [id_cliente]
    modo_carga: merge
    extraccion:
      modo: completa
      columna_marca: fecha_cambio
      solape_segundos: 5
    contrato: contracts/silver/clientes_crm.yml
```

**`clave_negocio`**

Lo que identifica una fila: deduplica y hace el merge

**`modo_carga`**

append · overwrite · merge (idempotente)

**`extraccion`**

completa o incremental sobre columna_marca: lo vemos en la demo 4

# La librería datafwk: el código que no se copia

| | |
|---|---|
| `arranque` | Contexto del notebook desde 2 parámetros |
| `rutas` | Única fuente de rutas de OneLake |
| `ingesta` | Landing → bronze, append-only |
| `estandarizar` | snake_case y deduplicación |
| `calidad` | Aplica el contrato y separa la cuarentena |
| `auditoria` | Columnas técnicas y ejecuciones |
| `escritura` | Delta: append, overwrite o merge |

## Así queda un notebook

```
ctx = cargar_contexto(
    entorno, workspace_datos,
    dominio)
a_silver(
    spark, ctx.fuente(fuente),
    ctx.contrato(fuente),
    ctx.entorno)
```

**FW009** Más de 25 líneas de código en un notebook: esa lógica va a src/datafwk/, con su test

# Sin trazabilidad no hay confianza

### `_ingested_at`

Cuándo llegó

### `_source`

De qué fuente

### `_batch_id`

En qué lote

Cada ejecución, una fila en fwk_ejecuciones; cada lote, en fwk_lotes; cada marca, en fwk_marcas. Y tiene que cuadrar:

**leídas = escritas + rechazadas**

Si no cuadra, la ejecución queda en estado no_reconcilia: se han perdido filas

DEMO 2 | 25 minutos | Lidera: Miguel

# Carga completa y dimensión tipo1

1. Abre crm.yml, el contrato silver y el modelo: marca fecha_cambio, completa, merge, tipo1
2. Bronze → silver → gold CRM con batch_id = crm-base
3. Esperado: 100 observaciones, 100 clientes y 100 filas en dim_cliente; C0001 en Sevilla
4. Repite los tres notebooks con crm-base: nada se duplica
5. En lh_silver: fwk_lotes confirmado, fwk_marcas con la marca UTC y fwk_ejecuciones

**La idea que se llevan**

**Un lote es el mismo identificador en bronze, silver y gold. Reintentar es repetir ese identificador.**

**Rama:** `alumno-99/demo-02`

**Stop session** entre notebooks: una sesión de Spark cada vez



# T3 Calidad, contratos y testing

Miguel Egea · 15 minutos · Demo 3

LECCIÓN 3

# El dato malo llega siempre. Lo único que decides es dónde aterriza.

En la cuarentena, con su motivo, o en el informe de dirección, sin que nadie lo vea venir.

La cuenta Miguel: una batalla real

# El contrato de datos: qué es un dato válido

```yaml
tabla: lineas_devolucion
capa: silver
propietario: "@org/datos-workshop"
clave: [id_linea_devolucion]
politica_rechazo: cuarentena
columnas:
  id_linea_devolucion: {tipo: string, nulo: false}
  importe_reembolso: {tipo: "decimal(18,2)",
    nulo: false}
  motivo: {tipo: string, nulo: true}
reglas:
  - nombre: reembolso_no_negativo
    expresion: importe_reembolso >= 0
version: 1
```

### Lo leen tres

Personas, la librería y la IA: un único fichero

### Lo aprueba negocio

CODEOWNERS: el dueño del dato firma cada cambio

### Se valida solo

JSON Schema en el editor, en pre-commit y en CI

# ¿Cuarentena o fallar?



**`politica_rechazo: cuarentena`**

- Aparta las filas inválidas y sigue la carga
- Para un ruido acotado y conocido
- Habitual en silver
- La evidencia va a fwk_cuarentena, por versión de contrato



**`politica_rechazo: fallar`**

- Detiene la carga entera
- Cuando el dato malo llegaría a una decisión
- Habitual en gold
- Demo 3, Q3: «39 filas incumplen el contrato; no se publica»

**Criterio:** ¿puede el dato malo llegar a una decisión de negocio? Si sí, se para.

# Tres niveles de test, y solo tres

1



## Unitarios

Funciones puras con DataFrames pequeños

pytest + Spark local · en cada PR

2



## De contrato

Tipos, nulos, valores y reglas

En cada ejecución, en Fabric

3



## De reconciliación

Bronze = silver + cuarentena

Entre capas, en cada lote

# Funciones puras y un framework mínimo

## Una función pura se prueba en local

```
def test_normalizar_tienda(spark):
    df = spark.createDataFrame(
        [("MAD1",), ("mad01",), ("XYZ",)],
        ["tienda"])
    assert valores(normalizar(df)) == [
        "MAD01", "MAD01", None]
```

Sin Fabric, sin capacidad, en segundos: por eso entra en cada PR

## ¿Y Great Expectations, pandera, Soda?



### Menos dependencias

Un módulo de la librería, no otra plataforma que mantener



### Un solo fichero

El contrato que aplica la librería es el mismo que lee la IA



### Decisión reversible

Si el proyecto lo pide, se cambia con un ADR

DEMO 3 40 minutos Lidera: Miguel

# Errores de origen y cuarentena Q1 → Q4

1. defectos y datos; bronze → silver devoluciones con dev-q1: 387 válidas, 10 rechazadas
2. checkpoint calidad (Q2: motivo obligatorio); revalida con dev-q2: 358 válidas, 39 rechazadas
3. checkpoint fallar (Q3): dev-q3 falla con «39 filas incumplen el contrato; no se publica»
4. checkpoint recuperar (Q4): repite el mismo dev-q3; confirma con 358 y 39
5. En cada paso: git diff del contrato y fwk_cuarentena agrupada por versión

La idea que se llevan

**Dato inválido no es regla que cambia. La evidencia se guarda por versión de contrato; el lote fallido se recupera, no se salta.**

**Rama:** `alumno-99/demo-03` · `independiente de CRM`




# Descanso · 15 minutos

Volvemos a las 18:25 con incremental e historia



# Demo 4·5 Del incremental a la historia

Eladio Rincón y Miguel Egea · 75 minutos

# Incremental: marca, ventana y solape

**`marca confirmada`**

Límite superior del último lote confirmado, en UTC

**`solape: 5 s`**

La ventana empieza 5 s antes de la marca

**`estabilización`**

5 s: lo más reciente se deja para la siguiente ventana

Cada lote deja su ventana en fwk_lotes. En CSV no hay pushdown: se lee todo y se filtra.

$$\text{ventana} = [\ \text{marca} - 5\ \text{s}\ ,\ \text{ahora} - 5\ \text{s}\ )$$

No se detectan borrados ni cambios intermedios. Devoluciones sigue en completa: no tiene marca fiable.

DEMO 4 | 30 minutos | Lidera: Eladio

# De carga completa a incremental

1. checkpoint incremental: un diff de una línea en crm.yml (modo: incremental)
2. modificar_csv ciudad --valor Barcelona y datos; espera 6 segundos
3. crm-inc-1: una fila en la ventana; C0001 pasa a Barcelona; siguen 100 clientes
4. Repite crm-inc-1: silver y gold en «reutilizado»; la marca no cambia
5. Tras 11 s, crm-inc-2 sin tocar el CSV: cero filas; el límite puede avanzar

**La idea que se llevan**

**Extracción y escritura son decisiones independientes. En CSV se lee el fichero entero y se filtra: ni pushdown ni CDC.**

**Comprueba en lh_silver:** *fwk_lotes (ventanas con solape de 5 s) y fwk_ejecuciones (batch_id LIKE 'crm-inc%')*

# De tipo1 a SCD2: la historia cambia el grano

## tipo1 · hasta ahora

- Una fila por id_cliente
- Un cambio de ciudad sobrescribe el valor
- Sevilla desaparece sin dejar rastro
- Clave: id_cliente

## scd2 · tras la activación

- Una fila por versión: id_version
- vigente_desde, vigente_hasta, es_actual
- A → B → A: tres versiones, una vigente
- dim_cliente_actual: la proyección para consumidores

**Cambio de grano.** Quien una hechos por id_cliente contra el histórico multiplica filas: 102 en lugar de 100. La historia empieza en la activación; no se reconstruye.

DEMO 5 | 45 minutos | Lidera: Miguel

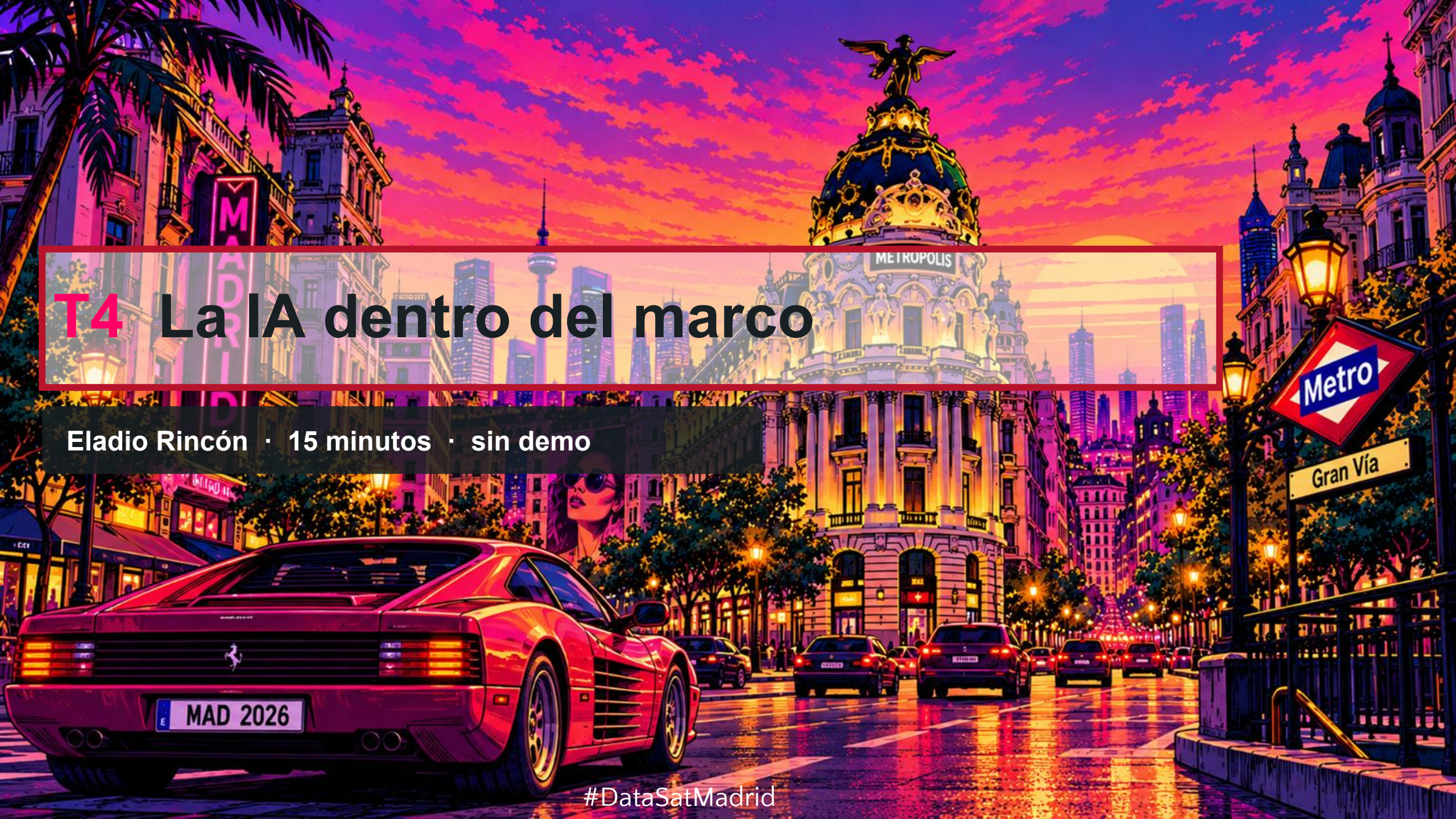
# SCD2: ciudad con historia

1. checkpoint scd2 y git diff contracts/gold config/modelos: la clave pasa de id_cliente a id_version
2. crm-activar: bronze → silver y nb_90_ops_activar_scd2; 100 versiones, 100 actuales
3. Valencia → crm-scd2-b con el gold ordinario; después Barcelona → crm-scd2-a
4. Esperado: C0001 con tres versiones y una vigente; 102 históricas, 100 en dim_cliente_actual
5. Consumo: actual ⋈ histórico por id_cliente da 102 filas; con es_actual = 1, vuelven a ser 100

**La idea que se llevan**

**Activar SCD2 es una migración explícita y un cambio de grano para quien consume. La historia empieza en la activación.**

**Rama:** *alumno-99/demo-05 · 13 notebooks: Stop session antes de cada uno*




# T4 La IA dentro del marco

Eladio Rincón · 15 minutos · sin demo

LECCIÓN 4

> **Cada generación de herramientas prometió que ya no harían falta estándares. Ninguna lo cumplió, y esta tampoco.**

Generar código rápido nunca fue el problema. Mantener lo generado, sí.

La cuenta Eladio: una batalla real

# La IA amplifica lo que encuentra

## Sin marco

- Nombres de tabla y columna a su manera
- Rutas abfss:// escritas a mano
- Se inventa qué es «tasa de devolución»
- Sin test ni contrato

## Con marco

- Respeta convenciones y plantillas
- Usa datafwk.rutas
- Busca la definición en el glosario o pregunta
- Contrato, test y catálogo incluidos

**Mismo modelo, misma petición.** Lo único que cambia es lo que encuentra en el repositorio.

# Ingeniería de contexto: lo que el asistente necesita



**Reglas**

`AGENTS.md`



**Definiciones**

`docs/glosario.md`



**Plantillas**

`templates/`



**Tareas repetibles**

`.github/prompts/`



**Criterios**

`docs/checklist-revision.md`

**El fichero que más falta en los proyectos es el glosario.** Si una métrica no está definida, la regla es preguntar, no inventar.

# AGENTS.md: reglas que se leen… y se comprueban

```
## Reglas no negociables
1. Toda tabla nueva necesita su contrato
   ANTES que el notebook.
2. Los notebooks se crean copiando
   templates/notebook/, con logicalId nuevo.
3. Nunca rutas abfss:// escritas a mano.
4. Nunca credenciales ni IDs en el código.
5. Toda transformación nueva lleva test.
6. Si una definición de negocio no está
   en docs/glosario.md, PREGUNTA.
```



### Jerárquico

Raíz + fabric/ingenieria + fabric/analitica



### Generado

La sección de linters sale de los propios linters



### Comprobado

Un test falla si se desfasa del código

# El agente pasa los linters mientras escribe



### El agente edita

Crea o modifica un fichero



### Salta el hook

PostToolUse tras cada edición



### agente_lint.py

Los linters de ese tipo de fichero



### Error concreto

Fichero, línea y regla



### Corrige y sigue

Antes de pasar a lo siguiente

### Claude Code

Automático: hook en .claude/settings.json

### Copilot

Por instrucción + copilot-setup-steps.yml

### Cualquier otro

AGENTS.md y los prompts del repositorio

# Los mismos linters en cada momento

**0**



### Mientras el agente escribe

agente_lint.py sobre el fichero que acaba de tocar

**1**



### En el editor

Extensiones de VS Code con la misma configuración

**2**



### Al hacer commit

pre-commit sobre los ficheros del commit

**3**



### En el pull request

CI sobre todo el repositorio: el que manda

Gitleaks · Ruff · mypy · reglas FW · yamllint · JSON Schema · SQLFluff · actionlint · zizmor · Terraform · BPA · Fab Inspector

# Dónde sigue siendo imprescindible la persona

### Decidir el modelo
Qué tablas, con qué grano y para quién

### Validar el negocio
Que la regla y la métrica son las correctas

### Revisar lo que no hizo
Lo que falta no lo marca ningún linter

### Aprobar los contratos
Lo que significa un dato lo firma su dueño

**La IA acelera la ejecución. El criterio sigue siendo humano.**


Cierre Lo que nos llevamos

# Cada lección, una pieza del framework

| Lección | Pieza del framework |
|---|---|
| El proyecto se decide en la primera semana | Convenciones + estructura |
| Lo que no está parametrizado se copia, y lo que se copia diverge | Librería + metadatos |
| Sin trazabilidad no hay confianza | Columnas técnicas + auditoría |
| El dato malo llega siempre; decide dónde aterriza | Contratos + cuarentena |
| Si no se puede desplegar sin ti, no está terminado | CI/CD |
| La IA amplifica lo que encuentra | AGENTS.md + plantillas + linters |

# El lunes, empezad por la primera pieza

## Escribid vuestras convenciones en un fichero.

Es lo primero que va a leer la IA.

No hace falta adoptar las siete piezas a la vez. Cada una se apoya en la anterior, y la primera cuesta una tarde.



**Repositorio plantilla**

`plantilla-framework-datos-fabric`

# Para seguir leyendo

**The next generation of data platforms is the Data Mesh**
PayPal Tech · por qué los datos se organizan por dominio

**Integración de Git en Microsoft Fabric**
Microsoft Learn · cómo se versionan los items

**fabric-cicd**
Despliegue de items de Fabric desde un pipeline

**AGENTS.md**
Formato abierto de instrucciones para agentes de código

**Hooks de Claude Code**
Ejecutar los linters tras cada edición del agente

# ¿Preguntas?


DATA
SATURDAYS
MADRID
Recuerda quedarte hasta el final y disfrutar de las
DATABEERS
#DataSatMadrid